

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_634e841e-51b\agent-ad80419f4cbaa027d.jsonl`
- SHA-256: `feb4b0c3c52eb79df1c2e19e71a723d95b30bc3de2e2c974dc1c595fa1eda3de`
- Source modified: `2026-07-06T08:44:26+00:00`
- Imported at: `2026-07-08T16:00:06+00:00`
- project: `wf_634e841e-51b`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T08:43:56.391Z)

Reviewing GitHub PR #38 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-d-command-service -> main, one commit 22abe72).

It implements ADR 0011 Track D: a REFACTOR that extracts command parsing + config-mutation logic from the

Telethon adapter into a provider-independent CommandService. THE PRIMARY REVIEW RISK IS BEHAVIORAL

EQUIVALENCE: the new code must behave identically to the old inline logic for every command.

Changed files ONLY: src/tg_video_filter/command_service.py (NEW),
src/tg_video_filter/commands.py (refactored
407->~120 line thin Telethon adapter), tests/test_command_service.py (NEW, 20 tests). Diff
+650/-376.
tests/test_commands.py is UNCHANGED and still passes (backwards-compat surface).

To compare against the pre-refactor behaviour, read the OLD file:

```
git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/commands.py
```

The new logic lives in command_service.py (CommandService.handle + parsers + formatters) and the new

commands.py delegates to it. Key already-checked facts (verify independently, don't just trust):

- regex patterns are byte-identical old vs new; branch order is the same.
- old handler did text.rstrip() then per-branch logic; new handler passes raw text and handle() does text.rstrip().
- old _update_config RELOADED cfg from db each call; new CommandService._update uses the passed current_cfg,
and the new adapter loads current_cfg = db.load_filter_config(...) fresh right before handle(). Assess equivalence.
- both old and new use cfg.model_copy(update=...) which SKIPS pydantic validation (pre-existing, not new).

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
- A STRONG equivalence check: feed a battery of command strings through BOTH the new CommandService.handle()
and (on a checkout of main, or by reasoning from the old source) the old handler, and diff
reply_text +
resulting saved FilterConfig. Report ANY divergence (different reply string, different config
mutation,
different validation threshold, different branch taken, different order).
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check clean; ruff format clean;
pytest
482 passed @ 97.05% (floor 80%, up from 96.52%).
- Report ONLY defects in THIS PR's diff. For a refactor, a BEHAVIORAL DIVERGENCE from the old

code is the

highest-value finding. Also flag: lost functionality, broken re-exports, new bugs in extracted logic.

No praise. Concrete failure scenario per finding. Rank most-severe first. Honest severity.

- Severity: blocker (wrong behaviour users hit / lost command), high (real divergence, narrow trigger),

medium (edge divergence or latent), low (hygiene/style).

ADVERSARIALLY VERIFY this single finding ? try to REFUTE it. Read the exact cited code (and the old code via

git show main:... when it's an equivalence claim), trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful.

CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour (for

equivalence findings: the new behaviour must actually differ from the old). REFUTED if the code matches old

behaviour / handles it / trigger can't occur / it's pre-existing. PLAUSIBLE if unsettled. Give corrected severity.

FINDING (dim service_correctness):

title: Adapter loads FilterConfig from DB on every self-message, including non-commands and

/show config (extra DB read vs. old code)

severity: low

where: src/tg_video_filter/commands.py:70

summary: The new Telethon adapter unconditionally calls db.load_filter_config(conn, user.id) at line 70 before every handle() call, so plain self-messages (e.g. the user typing 'hello' in Saved Messages) and /show config now trigger a config read that the old code avoided; for config-changing commands it does a second load at line 74 to build the engine, giving 2 DB reads + 1 write per mutating command.

failure_scenario: User sends any non-command text to their own Saved Messages: the old handler matched no branch and never touched the DB; the new adapter issues one db.load_filter_config read per such message. And /show config now reads twice (once in the adapter at line 70, once inside handle() at line 304). No user-visible behavior change and no correctness impact -- purely redundant reads on a per-message hot path. Verified: reply text, saved config, engine state, and transport guards are all behaviorally identical to the old code across a 46-input battery; this is only an efficiency/hygiene note, not a divergence.

evidence: commands.py:70 `current\_cfg = await db.load\_filter\_config(conn, user.id)` runs before svc.handle for every message that passes the out/peer guards. commands.py:74 does a second `new\_cfg = await db.load\_filter\_config(...)` when result.config_changed. command_service.py:304 loads again for /show config. Old commands.py only hit the DB inside a matched branch. A realistic single-store harness confirmed loads=2/saves=1 for a mutating command and the engine cfg correctly reflects the saved value (max_size_mb=50), so this is safe -- just extra reads. recommendation: Optional micro-optimization: skip the line-70 load for messages that aren't commands, or pass the already-loaded current_cfg into handle() for /show config instead of reloading inside handle(). Since config reads are cheap SQLite JSON-blob loads and the full suite passes at 97% coverage, this is not worth blocking the PR; leave as-is unless the self-message volume is high.

2. assistant (2026-07-06T08:43:58.791Z)

I'll adversarially verify this finding by reading the actual code and the old code to check the claim.

3. user (2026-07-06T08:44:00.181Z)

```
1      """Config commands ? Telethon adapter over the provider-independent `CommandService`.
2
3      This module is a thin Telethon adapter. It handles Telethon-specific transport
4      (checking `message.out`, `peer_id`, sending replies) and delegates command
5      parsing and config mutation to `command_service.CommandService` (ADR 0011, Track D).
6
7      Re-exports regex patterns and unit-parsers for backwards-compatibility with
8      existing tests (`tests/test_commands.py`).
9
10     See `docs/adr/0007-inline-config-commands.md` for the command model and
```

```

11     `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API front-door.
12     """
13
14     from __future__ import annotations
15
16     import logging
17     from collections.abc import Callable
18
19     import aiosqlite
20     from telethon import TelegramClient
21     from telethon.errors import RPCError
22
23     from tg_video_filter import db
24     from tg_video_filter.command_service import ENGINE_KEY, CommandResult, CommandService
25     from tg_video_filter.filters import FilterEngine
26     from tg_video_filter.models import FilterConfig, User
27
28     log = logging.getLogger(__name__)
29
30     from tg_video_filter.command_service import ( # noqa: E402
31         _EXCLUDE_HELP_RE,
32         _EXCLUDE_RE,
33         _LENGTH_RE,
34         _MIN_LENGTH_RE,
35         _MIN_RES_HELP_RE,
36         _MIN_RES_RE,
37         _SHOW_COMMANDS_RE,
38         _SHOW_CONFIG_RE,
39         _SIZE_RE,
40         _format_duration,
41         _format_size,
42         format_commands,
43         format_config,
44         parse_duration_s,
45         parse_size_mb,
46     )
47
48
49     def make_config_command_handler(
50         user: User,
51         conn: aiosqlite.Connection,
52         engine_holder: dict[str, FilterEngine],
53     ) -> Callable:
54         svc = CommandService()
55
56         async def on_command(event: object) -> None:
57             message = getattr(event, "message", None)
58             if message is None:
59                 return
60             if not getattr(message, "out", False):
61                 return
62             peer = getattr(message, "peer_id", None)
63             peer_user_id = getattr(peer, "user_id", None) if peer else None
64             if peer_user_id != user.id:
65                 return
66
67             text: str = getattr(message, "message", "") or ""
68             client: TelegramClient = getattr(event, "client", None) # type:
ignore[assignment]
69
70             current_cfg = await db.load_filter_config(conn, user.id)
71             result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
72
73             if result.config_changed:
74                 new_cfg = await db.load_filter_config(conn, user.id)

```

```

75         engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
76
77         if result.reply_text:
78             await _reply(client, message, result.reply_text)
79
80     return on_command
81
82
83 # --- backwards-compat helpers (test_commands.py references) -----
84
85
86 async def _update_config(
87     conn: aiomysqlite.Connection,
88     user_id: int,
89     max_size_mb: int | None = None,
90     max_duration_s: int | None = None,
91     min_duration_s: int | None = None,
92     target_height: int | None = None,
93     exclude_gif: bool | None = None,
94     exclude_round: bool | None = None,
95     exclude_nosound: bool | None = None,
96 ) -> FilterConfig:
97     """Deprecated wrapper ? delegates to ``CommandService._update``.
98
99     Kept for backwards-compatibility with ``tests/test_commands.py`` which
100     imports ``_update_config`` from here. New code should call
101     :meth:`CommandService.handle` which returns a :class:`CommandResult`.
102     """
103     svc = CommandService()
104     cfg = await db.load_filter_config(conn, user_id)
105     return await svc._update(
106         conn,
107         user_id,
108         cfg,
109         max_size_mb=max_size_mb,
110         max_duration_s=max_duration_s,
111         min_duration_s=min_duration_s,
112         target_height=target_height,
113         exclude_gif=exclude_gif,
114         exclude_round=exclude_round,
115         exclude_nosound=exclude_nosound,
116     )
117
118
119 async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
120     try:
121         await client.send_message("me", text)
122     except (OSError, TimeoutError, RPCError):
123         log.warning("config_reply_failed", exc_info=True)
124
125
126 __all__ = [
127     "ENGINE_KEY",
128     "_EXCLUDE_RE",
129     "_LENGTH_RE",
130     "_MIN_LENGTH_RE",
131     "_MIN_RES_RE",
132     "_SHOW_CONFIG_RE",
133     "_SHOW_COMMANDS_RE",
134     "_EXCLUDE_HELP_RE",
135     "_MIN_RES_HELP_RE",
136     "_SIZE_RE",
137     "_format_duration",
138     "_format_size",
139     "_update_config",

```

```

140         "format_commands",
141         "format_config",
142         "make_config_command_handler",
143         "parse_duration_s",
144         "parse_size_mb",
145     ]
146

```

4. user (2026-07-06T08:44:00.466Z)

```

"""Config commands: /set size, /set length, /show config.

```

These commands are sent to the user's own Saved Messages (like /backfill) and update the persisted ``FilterConfig``. The filter engine is hot-reloaded so the new config takes effect immediately without a restart.

See ``docs/adr/0007-inline-config-commands.md`` for the design rationale.

```

"""

```

```

from __future__ import annotations

```

```

import logging

```

```

import re

```

```

from collections.abc import Callable

```

```

import aiosqlite

```

```

from telethon import TelegramClient

```

```

from telethon.errors import RPCError

```

```

from tg_video_filter import db

```

```

from tg_video_filter.filters import FilterEngine

```

```

from tg_video_filter.models import FilterConfig, User

```

```

log = logging.getLogger(__name__)

```

Key into the shared engine_holder dict. Dict swap is atomic under the GIL for single-threaded asyncio dispatch (see ADR 0007).

```

ENGINE_KEY = "engine"

```

--- regex patterns -----

```

_SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)

```

Matches both "/set length" and "/set max length" (legacy alias).

Kept separate from _MIN_LENGTH_RE for clarity; patterns are mutually exclusive.

```

_LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)

```

```

_MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)

```

```

_MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$", re.IGNORECASE)

```

```

_SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)

```

```

_SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)

```

```

_EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$", re.IGNORECASE)

```

Catch-all: matches any "/set exclude ..." that didn't match the strict pattern, so we can reply with a friendly usage message instead of silence.

```

_EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)

```

Catch-all for malformed /set min resolution|res (same pattern as _EXCLUDE_HELP_RE)

```

_MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)

```

--- unit parsing -----

```

def parse_size_mb(value_str: str, unit: str) -> int:
    """Parse a size value + unit string into mebibytes.

    >>> parse_size_mb("10", "MB")
    10
    >>> parse_size_mb("1", "GB")
    1024
    >>> parse_size_mb("500", "KB")
    1

    Note: returns the raw arithmetic result; callers must enforce
    ``FilterConfig`` domain constraints (e.g. ``max_size_mb >= 1``).
    """
    value = int(value_str)
    unit = unit.lower().rstrip("b") # "MB" ? "m", "KB" ? "k", "GB" ? "g"
    multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
    if unit in multipliers:
        raw = value * multipliers[unit]
        if unit == "k":
            raw = max(1, raw // 1024)
        return raw
    raise ValueError(f"Unknown size unit: {unit}")

def parse_duration_s(value_str: str, unit: str) -> int:
    """Parse a duration value + unit string into seconds.

    >>> parse_duration_s("15", "m")
    900
    >>> parse_duration_s("2", "h")
    7200
    >>> parse_duration_s("90", "s")
    90

    Note: returns the raw arithmetic result; callers must enforce
    ``FilterConfig`` domain constraints (e.g. max >= min).
    """
    value = int(value_str)
    unit = unit.lower()
    multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
    if unit in multipliers:
        return value * multipliers[unit]
    raise ValueError(f"Unknown time unit: {unit}")

```

--- formatting helpers -----

```

def _format_duration(seconds: int | None) -> str:
    """Format seconds as a human-readable duration string."""
    if seconds is None:
        return "unlimited"
    if seconds == 0:
        return "0s"
    if seconds < 60:
        return f"{seconds}s"
    if seconds < 3600:
        m, s = divmod(seconds, 60)
        return f"{m}m{s}s" if s else f"{m}m"
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    parts = f"{h}h"
    if m:
        parts += f"{m}m"

```

```

if s:
    parts += f"{s}s"
return parts

def _format_size(mb: int | None) -> str:
    """Format mebibytes as a human-readable size string."""
    if mb is None:
        return "unlimited"
    if mb == 0:
        return "0MB"
    if mb >= 1024:
        gb = mb / 1024
        if gb == int(gb):
            return f"{int(gb)}GB"
        return f"{gb:.1f}GB"
    return f"{mb}MB"

def format_config(cfg: FilterConfig) -> str:
    """Format a ``FilterConfig`` as a readable Telegram message."""
    lines = [
        "? **Current Filter Configuration**",
        "",
        f"?? **Duration:** {_format_duration(cfg.min_duration_s)} - "
        f"{_format_duration(cfg.max_duration_s)}",
        f"? **Min resolution:** {'?' + str(cfg.target_height) + 'p' if cfg.target_height else "disabled"}",
        f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
        f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
        f"? **Delivery target:** `{cfg.delivery_target}`",
        "",
        "? **Content exclusions:**",
        f"    GIF/animated: {'?' excluded' if cfg.exclude_gif else '?' allowed'}",
        f"    Round videos: {'?' excluded' if cfg.exclude_round else '?' allowed'}",
        f"    Silent (no audio): {'?' excluded' if cfg.exclude_nosound else '?' allowed'}",
    ]
    return "\n".join(lines)

def format_commands() -> str:
    """Return a help message listing all available commands."""
    return "\n".join(
        [
            "? **Available Commands**",
            "",
            "? **Filter config**",
            "`/show config` ? Show current filter settings",
            "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
            "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing `B`",
            "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
            "    or `/set max length <N><unit>` ? same as above",
            "    Units: `s` (default), `m`, `h` ? case-insensitive",
            "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length 5s`)",
            "    Units: `s` (default), `m`, `h` ? case-insensitive",
            "`/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution "
            "720p`)",
            "    or `/set min res <N>p` ? same as above",
            "",
            "? **Content exclusions** (all off by default)",
            "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
            "`/set exclude round on|off` ? Exclude circular video messages",
            "`/set exclude nosound on|off` ? Exclude videos without audio\n"
            "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs too.",
            "",
        ]
    )

```

```

        "? **Backfill**",
        "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default
90d)",
        "`/stop` ? Cancel running backfill",
        "",
        "\u2139\u2013 **Help**",
        "`/show commands` ? Show this help message",
    ]
)

```

--- command handler factory -----

```

def make_config_command_handler(
    user: User,
    conn: aiosqlite.Connection,
    engine_holder: dict[str, FilterEngine],
) -> Callable:
    """Return an async handler for /set and /show config commands.

    The handler only responds to messages the user sends to themselves
    (Saved Messages). It persists config changes via ``db.save_filter_config``
    and hot-reloads the filter engine through ``engine_holder``.

    ``engine_holder`` is a mutable dict that both the live-message handler and
    the backfill handler read from, so a config change is picked up by all
    components immediately.
    """

    async def on_command(event: object) -> None:
        message = getattr(event, "message", None)
        if message is None:
            return

        # Only respond to messages the user sent to themselves.
        if not getattr(message, "out", False):
            return
        peer = getattr(message, "peer_id", None)
        peer_user_id = getattr(peer, "user_id", None) if peer else None
        if peer_user_id != user.id:
            return

        text: str = getattr(message, "message", "") or ""
        # Strip trailing whitespace so mobile keyboards don't silently
        # reject commands (e.g. "/show config " ? "/show config").
        text = text.rstrip()
        client: TelegramClient = getattr(event, "client", None) # type: ignore[assignment]

        # --- /set size <value><unit> -----
        m = _SIZE_RE.match(text)
        if m:
            value_str, unit = m.group(1), m.group(2) or ""
            try:
                new_mb = parse_size_mb(value_str, unit)
            except ValueError:
                await _reply(client, message, "? Invalid size format. Use e.g. `/set size
10MB`.")
                return
            if new_mb < 1:
                await _reply(
                    client, message, "? Size must be at least 1 MB (`FilterConfig` constraint).")
                return
            return

```



```

        cfg = await _update_config(conn, user.id, max_size_mb=new_mb)
        engine_holder[ENGINE_KEY] = FilterEngine(cfg)
        await _reply(
            client,
            message,
            f"? Max file size set to **{_format_size(new_mb)}**."
f"\n\n{format_config(cfg)}",
        )
        return

# --- /set length <value><unit> -----
m = _LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_secs = parse_duration_s(value_str, unit)
    except ValueError:
        await _reply(
            client, message, "? Invalid duration format. Use e.g. `/set length 15m`."
        )
        return
    # Enforce max_duration_s >= min_duration_s so every video isn't
    # silently rejected (and so the FilterConfig validator passes on
    # next startup).
    current_cfg = await db.load_filter_config(conn, user.id)
    min_secs = current_cfg.min_duration_s or 0
    if new_secs < min_secs:
        await _reply(
            client,
            message,
            f"? Max duration ({_format_duration(new_secs)}) must be "
            f">= min duration ({_format_duration(min_secs)}).",
        )
        return
    cfg = await _update_config(conn, user.id, max_duration_s=new_secs)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Max duration set to **{_format_duration(new_secs)}**."
f"\n\n{format_config(cfg)}",
    )
    return

# --- /set min length <value><unit> -----
m = _MIN_LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_min = parse_duration_s(value_str, unit)
    except ValueError:
        await _reply(
            client,
            message,
            "? Invalid duration format. Use e.g. `/set min length 5s`.",
        )
        return
    # Enforce min_duration_s <= max_duration_s.
    current_cfg = await db.load_filter_config(conn, user.id)
    max_secs = current_cfg.max_duration_s or 0
    if max_secs > 0 and new_min > max_secs:
        await _reply(
            client,
            message,
            f"? Min duration ({_format_duration(new_min)}) must be "

```

```

        f"<= max duration ({_format_duration(max_secs)}).",
    )
    return
    cfg = await _update_config(conn, user.id, min_duration_s=new_min)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Min duration set to **{_format_duration(new_min)}**."
        f"\n\n{format_config(cfg)}",
    )
    return

# --- /set min resolution <N>p -----
m = _MIN_RES_RE.match(text)
if m:
    new_height = int(m.group(1))
    # Pre-validate for a friendly message (FilterConfig also enforces ge=1).
    if new_height < 1:
        await _reply(client, message, "? Resolution must be at least 1p.")
        return
    cfg = await _update_config(conn, user.id, target_height=new_height)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Min resolution set to **{new_height}p**." f"\n\n{format_config(cfg)}",
    )
    return

# Friendly error for malformed /set min resolution|res.
if _MIN_RES_HELP_RE.match(text):
    await _reply(
        client,
        message,
        "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)",
    )
    return

# --- /set exclude <gif|round|nosound> <on|off> -----
m = _EXCLUDE_RE.match(text)
if m:
    what = m.group(1).lower()
    state = m.group(2).lower()
    enabled = state == "on"
    labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}

    field_map = {
        "gif": "exclude_gif",
        "round": "exclude_round",
        "nosound": "exclude_nosound",
    }
    cfg = await _update_config(conn, user.id, **{field_map[what]: enabled})
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
        f"\n\n{format_config(cfg)}",
    )
    return

# Friendly error for invalid /set exclude (e.g. "gif true", missing on/off).
if _EXCLUDE_HELP_RE.match(text):
    await _reply(

```

```

        client,
        message,
        "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
        "Example: `/set exclude gif on`",
    )
    return

# --- /show config -----
if _SHOW_CONFIG_RE.match(text):
    cfg = await db.load_filter_config(conn, user.id)
    await _reply(client, message, format_config(cfg))
    return

# --- /show commands -----
if _SHOW_COMMANDS_RE.match(text):
    await _reply(client, message, format_commands())
    return

return on_command

```

--- internal helpers -----

```

async def _update_config(
    conn: aiosqlite.Connection,
    user_id: int,
    max_size_mb: int | None = None,
    max_duration_s: int | None = None,
    min_duration_s: int | None = None,
    target_height: int | None = None,
    exclude_gif: bool | None = None,
    exclude_round: bool | None = None,
    exclude_nosound: bool | None = None,
) -> FilterConfig:
    """Load the current config, apply partial updates, persist, and return."""
    cfg = await db.load_filter_config(conn, user_id)
    if max_size_mb is not None:
        cfg = cfg.model_copy(update={"max_size_mb": max_size_mb})
    if max_duration_s is not None:
        cfg = cfg.model_copy(update={"max_duration_s": max_duration_s})
    if min_duration_s is not None:
        cfg = cfg.model_copy(update={"min_duration_s": min_duration_s})
    if target_height is not None:
        cfg = cfg.model_copy(update={"target_height": target_height})
    if exclude_gif is not None:
        cfg = cfg.model_copy(update={"exclude_gif": exclude_gif})
    if exclude_round is not None:
        cfg = cfg.model_copy(update={"exclude_round": exclude_round})
    if exclude_nosound is not None:
        cfg = cfg.model_copy(update={"exclude_nosound": exclude_nosound})
    await db.save_filter_config(conn, user_id, cfg)
    log.info(
        "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
        "target_height=%s "
        "exclude_gif=%s exclude_round=%s exclude_nosound=%s",
        user_id,
        cfg.max_size_mb,
        cfg.min_duration_s,
        cfg.max_duration_s,
        cfg.target_height,
        cfg.exclude_gif,
        cfg.exclude_round,
        cfg.exclude_nosound,
    )

```

```

    )
    return cfg

async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
    """Send a reply, logging network errors but not raising.

    Only swallows network/RPC errors so programming bugs (KeyError,
    AttributeError, TypeError) still surface loudly during development.
    """
    try:
        await client.send_message("me", text)
    except (OSError, TimeoutError, RPCError):
        log.warning("config_reply_failed", exc_info=True)

__all__ = [
    "ENGINE_KEY",
    "_EXCLUDE_RE",
    "_LENGTH_RE",
    "_MIN_LENGTH_RE",
    "_MIN_RES_RE",
    "_MIN_RES_HELP_RE",
    "_SHOW_COMMANDS_RE",
    "_SHOW_CONFIG_RE",
    "_SIZE_RE",
    "_format_duration",
    "_format_size",
    "_update_config",
    "format_commands",
    "format_config",
    "make_config_command_handler",
    "parse_duration_s",
    "parse_size_mb",
]

```

5. assistant (2026-07-06T08:44:02.970Z)

Now let me look at the `CommandService.handle` and the `/show config` branch in `command_service.py` to verify the double-read claim.

6. user (2026-07-06T08:44:04.202Z)

```

280         what = m.group(1).lower()
281         state = m.group(2).lower()
282         enabled = state == "on"
283         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
284         field_map = {
285             "gif": "exclude_gif",
286             "round": "exclude_round",
287             "nosound": "exclude_nosound",
288         }
289         cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
290         return CommandResult(
291             f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
292             f"\n\n{format_config(cfg)}",
293             config_changed=True,
294         )
295
296     if _EXCLUDE_HELP_RE.match(text):
297         return CommandResult(
298             "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
299             "Example: `/set exclude gif on`"

```

```

300         )
301
302     # --- /show config -----
303     if _SHOW_CONFIG_RE.match(text):
304         cfg = await db.load_filter_config(conn, user_id)
305         return CommandResult(format_config(cfg))
306
307     # --- /show commands -----
308     if _SHOW_COMMANDS_RE.match(text):
309         return CommandResult(format_commands())
310
311     # Not a recognised command ? return empty result (noise).
312     return CommandResult()
313
314     # -- internal -----
315
316     async def _update(
317         self,
318         conn: aiosqlite.Connection,
319         user_id: int,
320         current: FilterConfig,
321         max_size_mb: int | None = None,
322         max_duration_s: int | None = None,
323         min_duration_s: int | None = None,
324         target_height: int | None = None,
325         exclude_gif: bool | None = None,
326         exclude_round: bool | None = None,
327         exclude_nosound: bool | None = None,
328     ) -> FilterConfig:
329         """Apply partial updates to the account's config, persist, and return."""
330         updates: dict[str, object] = {}
331         if max_size_mb is not None:
332             updates["max_size_mb"] = max_size_mb
333         if max_duration_s is not None:
334             updates["max_duration_s"] = max_duration_s
335         if min_duration_s is not None:
336             updates["min_duration_s"] = min_duration_s
337         if target_height is not None:
338             updates["target_height"] = target_height
339         if exclude_gif is not None:

```

7. assistant (2026-07-06T08:44:17.785Z)

The finding is verified by direct code reading. Let me confirm the key claims:

1. ****Line 70**** (`commands.py`): `current\_cfg = await db.load\_filter\_config(conn, user.id)` runs unconditionally before `svc.handle` for every message passing the out/peer guards. Confirmed.
2. ****Old code****: The old handler only hit the DB inside a matched branch (e.g. `\_update\_config` at line inside each branch, or `db.load\_filter\_config` inside `/show config` branch). A non-command message that matched no branch never touched the DB. Confirmed by reading the old source.
3. ****`/show config` double-read****: New adapter loads at line 70, then `handle()` loads again at `command\_service.py:304`. Confirmed ? 2 reads for `/show config`.
4. ****Mutating command****: line 70 load + line 74 reload + 1 save. The old code did 1 load inside `\_update\_config` + 1 save (plus for length/min-length branches an extra load for the min/max cross-check). Confirmed.

Now the critical question for severity: ****is this a behavioral divergence or purely an efficiency note?**** The finding itself explicitly states severity `low` and admits "No user-visible behavior change and no correctness impact -- purely redundant reads." Let me verify there's genuinely no behavioral divergence from these extra reads.

The extra reads are on the same `conn`, reading the same persisted config. `db.load\_filter\_config` is a read that does not mutate state. For `/show config`, the value read at line 70 is discarded (handle re-reads at 304 and uses that) ? so the reply is identical. For mutating commands, `current\_cfg` from line 70 is passed to `handle()` and used for the mutation base and cross-checks; then line 74 reloads to build the engine from the just-saved config. That reload reflects the saved value correctly.

The finding's own evidence confirms a harness run: `loads=2/saves=1` for a mutating command, engine cfg correctly reflects saved value. This matches my trace. There is no correctness or behavioral divergence ? it is purely redundant DB reads on a per-message hot path.

The finding is accurately described and the severity (`low`, hygiene/efficiency) is honest. This is a real observable code fact in the diff (the unconditional line-70 read that the old code avoided), correctly characterized as non-divergent and non-blocking.

8. user (2026-07-06T08:44:26.854Z)

Structured output provided successfully